

TRACE

TRACKING, REPORTING, ANALYSIS & COMMUNITY EVIDENCE

TRACE

Tracking, Reporting, Analysis & Community Evidence

Community vehicle tracking platform for neighborhood safety chapters. Reporters submit sightings from their phones. Operators triage, dispatch, and build intelligence from a desktop console. Three-vault database architecture ensures reporter identities stay separate from operational data.

[\[!Deploy with Vercel\]\(https://vercel.com/button\)](https://vercel.com/button)[\[https://vercel.com/new/clone?repository-url=https%3A%2F%2Fgithub.com%2Fduke-of-beans%2FTRACE&env=DATABASE_URL&envDescription=Neon%20PostgreSQL%20connection%20string%20\(pooler%20URL\)&envLink=https%3A%2F%2Fneon.tech&project-name=trace&repository-name=TRACE\]](https://vercel.com/new/clone?repository-url=https%3A%2F%2Fgithub.com%2Fduke-of-beans%2FTRACE&env=DATABASE_URL&envDescription=Neon%20PostgreSQL%20connection%20string%20(pooler%20URL)&envLink=https%3A%2F%2Fneon.tech&project-name=trace&repository-name=TRACE))

You need a computer with a web browser. That's it. Everything runs in the cloud for free.

Step 1: Create your database

1. Go to [\[neon.tech\]\(https://neon.tech\)](https://neon.tech) and sign up (you can use your Google account)
2. Click **New Project**
3. Name it anything (like ``trace``)
4. After it's created, you'll see a **Connection String** box. Click the copy button next to it. It looks something like ``postgresql://neondb_owner:abc123@ep-something.neon.tech/neondb?sslmode=require``

Keep this copied — you'll need it in the next step.

Step 2: Deploy TRACE

1. Click the blue **Deploy with Vercel** button above
2. Sign in with GitHub (create a free account if you don't have one)
3. It asks for ``DATABASE_URL`` — paste the connection string you just copied
4. Click **Deploy** and wait about 2 minutes

When it finishes, you'll get a URL like `trace-abc123.vercel.app`. This is your TRACE. Bookmark it.

Step 3: Set up the database tables

1. Go back to [Neon](https://console.neon.tech) and click on your project
2. Click **SQL Editor** in the left sidebar
3. Open the file setup.sql from this repository (click it on GitHub, then click the copy button)
4. Paste the entire contents into the Neon SQL Editor
5. Click **Run**

You should see a series of "CREATE TABLE" confirmations. If you see any errors about things "already existing," that's fine — it means the tables were already created.

Step 4: Create your operator account

1. Go to `your-app.vercel.app/operator/` (replace with your actual Vercel URL)`
2. You'll see a **First-Time Setup** screen
3. Enter your **chapter name** (e.g., "Westside Watch")
4. Choose a **callsign** (your operator name — like "ALPHA" or "DISPATCH-1")
5. Choose an **access code** (like a password — 6+ characters, keep it safe)
6. Click **Create Operator & Start**

You're in. The onboarding walkthrough will show you around.

Step 5: Invite your first reporter

1. In the operator console, click **Admin** in the sidebar
2. Click the **Reporters** tab
3. Type a callsign for the reporter (their code name, not their real name)
4. Click **Generate Invite Code**
5. Send the invite code to your reporter via Signal, WhatsApp, or in person

The reporter opens your TRACE URL on their phone, enters the code, and they're in.

Step 6: Install the app on phones

Tell reporters to open your TRACE URL in their phone's browser, then:

iPhone (must use Safari): Tap the Share button → Add to Home Screen → Add

Android (use Chrome): Tap the three-dot menu → Add to Home Screen → Add

The app icon appears on their home screen and runs full-screen like a real app.

For reporters

Open the app. Enter your invite code. You see a plate input and a map.

Report mode. Type a plate, describe the activity, drop the location, submit. Takes 15 seconds. The operator sees it immediately in their triage queue.

Check Plate mode. Type a plate to see if it is already in the database. If it matches, you can escalate to a full report with one tap.

Map tab. See active dispatch pins dropped by the operator. Tap a pin to respond. The operator sees your status (responding, on scene).

For operators

The operator console is a desktop application at </operator/>.

Triage. Incoming sightings appear in a queue. Each one shows whether the plate matches a known vehicle (MATCH badge) or is new (NEW PLATE). Confirm and dispatch, dismiss with feedback to the reporter, or add the vehicle to tracking.

Intel Map. All sightings on a satellite map. Right-click to drop a dispatch pin. Click any sighting marker to see details. Time playback shows sighting patterns hour by hour. Corridor overlays trace vehicle movement paths.

Vehicles and Actors. Dossier pages for tracked vehicles and persons of interest. Photos, suspicion levels, sighting history, identifier records.

Admin. Configure vehicle types, suspicion ladders, dispatch event types, actor identifiers. Generate reporter invite codes.

Three-vault database

TRACE separates data into three PostgreSQL schemas. If someone gains access to the operational data (vehicles, sightings, actors), they cannot determine who submitted any report. Reporter real identities live in a separate schema with a separate encryption key.

Schema	Contains	Access
`ops`	Vehicles, sightings, actors, dispatches, chapters	Full CRUD
`ident`	Reporter identities, auth tokens, sessions	Auth service only
`evidence`	Write-once evidence locker, hash chain	INSERT + SELECT only

A full dump of the [ops](#) schema reveals zero real identities. By architecture, not policy.

Stack

- **API:** Hono (TypeScript) on Vercel Serverless Functions
- **Database:** Neon PostgreSQL + Drizzle ORM
- **Reporter app:** Preact PWA (mobile-first, installable)
- **Operator console:** React SPA (desktop-first)
- **Maps:** Leaflet.js with Esri/CARTO/OSM tile layers
- **Auth:** Invite codes (no email required), TOTP for operators

Project structure

```
src/  
  api/  
    auth/      Invite-code and dev-login auth  
    sightings/ Sighting intake + plate matching  
    vehicles/   Vehicle dossier CRUD  
    actors/     Actor profile management  
    dispatch/   Dispatch event lifecycle  
    geo/        Heatmap, corridors, co-occurrence, temporal  
    admin/      Chapter configuration + reporter invites  
  db/  
    schema/     Drizzle schema (three vaults)  
    connection.ts Three connection pools  
    seed.ts     Demo data  
    services/    Plate lookup, geospatial, notifications  
  
  pwa/  
    src/pages/   Report, Map, History, Settings tabs  
    src/components/ Onboarding, panic button, pin lock  
  
  operator/  
    src/pages/   Dashboard, Triage, Intel, Vehicles, Actors, Admin, Security  
    src/components/ Map, time slider, UX primitives  
  
api/index.ts    Vercel serverless entry point  
shared/design/  Design tokens and theme
```

Authentication model

Reporters authenticate with a one-time invite code generated by an operator. No email, no phone number, no password. The invite code is given in person or via an encrypted channel (Signal). Once used, the code is invalidated.

Operators authenticate with a callsign and access code. The first operator is created during initial setup (see Quick Deploy step 5).

Production hardening checklist

Before giving anyone access to your TRACE instance:

- Set `TRACE_DISABLE_DEV_LOGIN=true` in Vercel env vars
- Set `TRACE_DISABLE_TEST_CODE=true` in Vercel env vars
- Change the default operator callsign from `OPERATOR` to something unique
- Verify the operator console rejects non-operator logins (it does by default)
- All access is logged to the audit table

Who can see what

Role	Sees	Does not see
Reporter	Their own submissions, active dispatch pins, plate check results	Other reporters, operator actions, vehicle dossiers, actor profiles
Operator	All sightings (by callsign, not real name), vehicles, actors, dispatches	Reporter real identities, Vault B data
Database admin	All three schemas	Nothing is hidden from the database admin

Reporter safety

The reporter app includes a panic button (configurable in Settings) and a PIN lock. The operator can remotely suspend a reporter's device via the Security page, which sends a kill signal on the next status check.

Prerequisites

- Node.js 22+
- PostgreSQL 15+ (local, Docker, or WSL)

Setup

```
git clone https://github.com/duke-of-beans/TRACE.git
cd TRACE
npm install
cd pwa && npm install && cd ..
cd operator && npm install && cd ..
cp .env.example .env
# Edit .env with your local PostgreSQL credentials
```

Database setup

```
# Create the database
psql -U postgres -c "CREATE DATABASE trace;"

# Run migrations
npx drizzle-kit migrate

# Seed demo data
npx tsx src/db/seed.ts
```

Run

```
# API server (port 3100)
npm run dev

# Reporter PWA (port 5173)
cd pwa && npm run dev

# Operator console (port 5174)
cd operator && npm run dev
```

Demo credentials: operator callsign **OPERATOR**, reporter invite code **TEST-CODE**.

TRACE is a Progressive Web App. No app store needed.

iPhone (Safari only)

1. Open your TRACE URL in Safari
2. Tap the Share button (square with arrow)
3. Scroll down, tap "Add to Home Screen"
4. Tap "Add"

The app appears on the home screen with a standalone icon. It runs full-screen without Safari's address bar.

Android (Chrome)

1. Open your TRACE URL in Chrome
2. Tap the three-dot menu
3. Tap "Add to Home Screen" or "Install App"
4. Tap "Add"

Desktop (Chrome/Edge)

1. Open your TRACE URL
2. Click the install icon in the address bar (or three-dot menu > Install)

- [Chapter Setup Guide](docs/CHAPTER_SETUP.md) — complete deployment walkthrough for new chapters
- [Dispatch System Design](DISPATCH_DESIGN.md)
- [Voice Guide](VOICE_GUIDE.md)
- [Design System](DESIGN_SYSTEM.md)
- [Security Architecture](docs/SECURITY_ARCHITECTURE.md)
- [Security Edge Cases](docs/SECURITY_EDGE_CASES.md)
- [Requirements Synthesis](docs/REQUIREMENTS_SYNTHESIS.md)

Source-available. Contact for chapter licensing.